Introduction to Statistical Learning with Applications

CM6: Linear classifiers – Part 2

Pedro L. C. Rodrigues

- Review from last lecture
- Generative approach to classification
- Comparisons between methods
- Final remarks on classification

The Bayes classifier

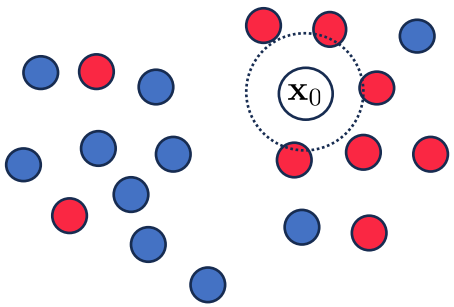
We want to find a function $f : \mathbb{R}^p \to \{0, 1\}$ such that it minimizes the probability error

$$\mathcal{L}(f) = \text{Prob}\Big(f(X) \neq Y\Big) = \mathbb{E}_{\mathbf{X},Y}\Big[\mathbf{1}_{f(\mathbf{X})\neq Y}\Big]$$

The optimal discriminant function for a test observation $\mathbf{x}_0$ is the Bayes classifier

$$\boxed{f(\mathbf{x}_0) = \underset{k\in\{0,1\}}{\text{argmax}}\ \text{Prob}\Big(Y = k \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big)}$$

For example,



$$\text{Prob}\Big(Y = \blacksquare \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) = 0.80$$

$$\text{Prob}\Big(Y = \blacksquare \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) = 0.20$$

$$f(\mathbf{x}_0) = \blacksquare$$

The Bayes classifier

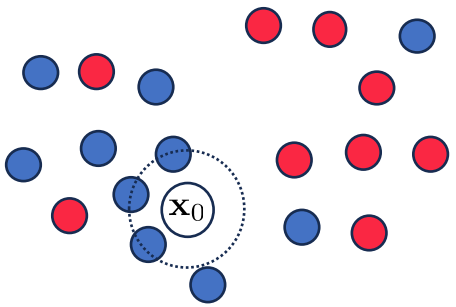
We want to find a function $f : \mathbb{R}^p \to \{0, 1\}$ such that it minimizes the probability error

$$\mathcal{L}(f) = \text{Prob}\Big(f(X) \neq Y\Big) = \mathbb{E}_{\mathbf{X},Y}\Big[\mathbf{1}_{f(\mathbf{X})\neq Y}\Big]$$

The optimal discriminant function for a test observation $\mathbf{x}_0$ is the Bayes classifier

$$\boxed{f(\mathbf{x}_0) = \underset{k\in\{0,1\}}{\text{argmax}}\ \text{Prob}\Big(Y = k \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big)}$$

For example,



$$\text{Prob}\Big(Y = \blacksquare \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) = 0.20$$

$$\text{Prob}\Big(Y = \blacksquare \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) = 0.80$$

$$f(\mathbf{x}_0) = \blacksquare$$

The Bayes classifier

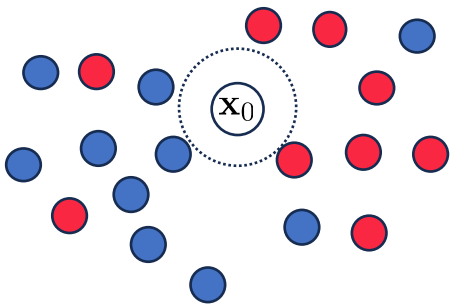
We want to find a function $f : \mathbb{R}^p \to \{0, 1\}$ such that it minimizes the probability error

$$\mathcal{L}(f) = \text{Prob}\Big(f(X) \neq Y\Big) = \mathbb{E}_{\mathbf{X},Y}\Big[\mathbf{1}_{f(\mathbf{X})\neq Y}\Big]$$

The optimal discriminant function for a test observation $\mathbf{x}_0$ is the Bayes classifier

$$\boxed{f(\mathbf{x}_0) = \underset{k\in\{0,1\}}{\text{argmax}}\ \text{Prob}\Big(Y = k \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big)}$$

For example,



$$\text{Prob}\Big(Y = \blacksquare \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) = 0.50$$

$$\text{Prob}\Big(Y = \blacksquare \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) = 0.50$$

$$f(\mathbf{x}_0) = ?$$

The Bayes classifier

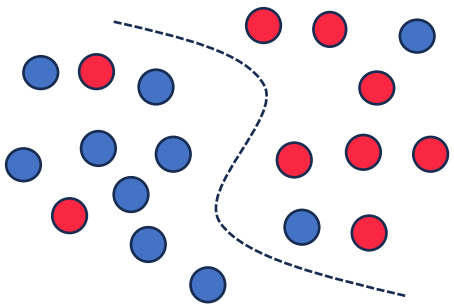
We want to find a function $f : \mathbb{R}^p \to \{0, 1\}$ such that it minimizes the probability error

$$\mathcal{L}(f) = \text{Prob}\Big(f(X) \neq Y\Big) = \mathbb{E}_{\mathbf{X},Y}\Big[\mathbf{1}_{f(\mathbf{X})\neq Y}\Big]$$

The optimal discriminant function for a test observation $\mathbf{x}_0$ is the Bayes classifier

$$\boxed{f(\mathbf{x}_0) = \underset{k\in\{0,1\}}{\text{argmax}}\ \text{Prob}\Big(Y = k \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big)}$$

For example,



$$\text{Prob}\Big(Y = \blacksquare \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) = 0.50$$

$$\text{Prob}\Big(Y = \blacksquare \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) = 0.50$$

$$f(\mathbf{x}_0) = ?$$

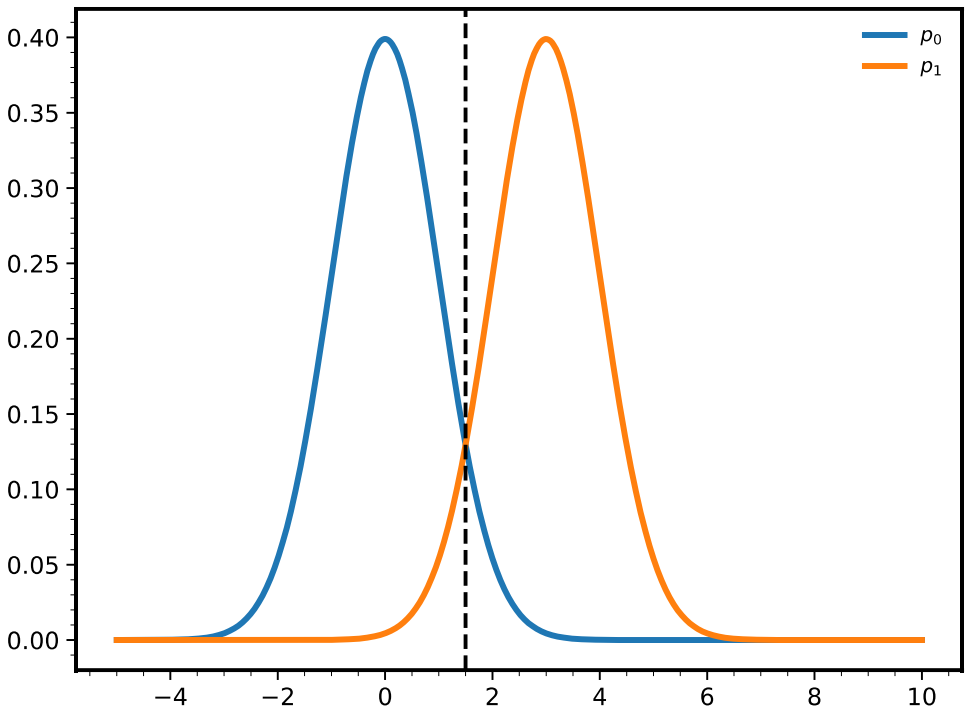
The Bayes classifier

Example: Consider the simple case in which

$$X \mid Y = 0 \sim \mathcal{N}(m_0, 1) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(x - m_0)^2\right)$$

$$X \mid Y = 1 \sim \mathcal{N}(m_1, 1) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(x - m_1)^2\right)$$

$$\text{Prob}(Y = 0) = \text{Prob}(Y = 1) = \frac{1}{2}$$



Show that the Bayes classifier is $f(\mathbf{x}_0) = \mathbf{1}_{\mathbf{x}_0 \geq c}$ where $c = \frac{1}{2}(m_0 + m_1)$

The Bayes classifier

$$f(\mathbf{x}_0) = \underset{k}{\operatorname{argmax}} \operatorname{Prob}\Big(Y = k \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big)$$

The problem of classification can be split into two separate stages

Use training data to learn a model for

$$\operatorname{Prob}\Big(Y = k \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x})\Big)$$

Use the learned posterior to make optimal class assignments

Discriminative approaches to classification approximate the posterior distribution

Generative approaches aim at modeling the full probability distribution of the data

Classification

Suppose we want to learn how to classify photos of dogs versus elephants

Classification

Suppose we want to learn how to classify photos of dogs versus elephants

Discriminative approaches learn how to build the boundary

Discriminative approaches

- **k-nearest neighbours (kNN)**

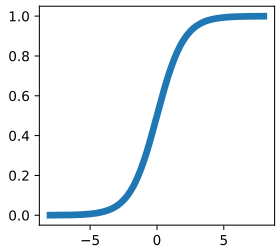
In kNN we approximate $\mathcal{B}_\varepsilon(\mathbf{x}_0) \approx \mathcal{N}_K(\mathbf{x}_0)$ i.e. the set of K nearest neighbours of $\mathbf{x}_0$

$$\mathrm{Prob}\Big(Y = 1 \;\Big|\; \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) \approx \frac{1}{K} \sum_{\mathbf{x}_i \in \mathcal{N}_K(\mathbf{x}_0)} \mathbf{1}_{y=1}(y_i)$$

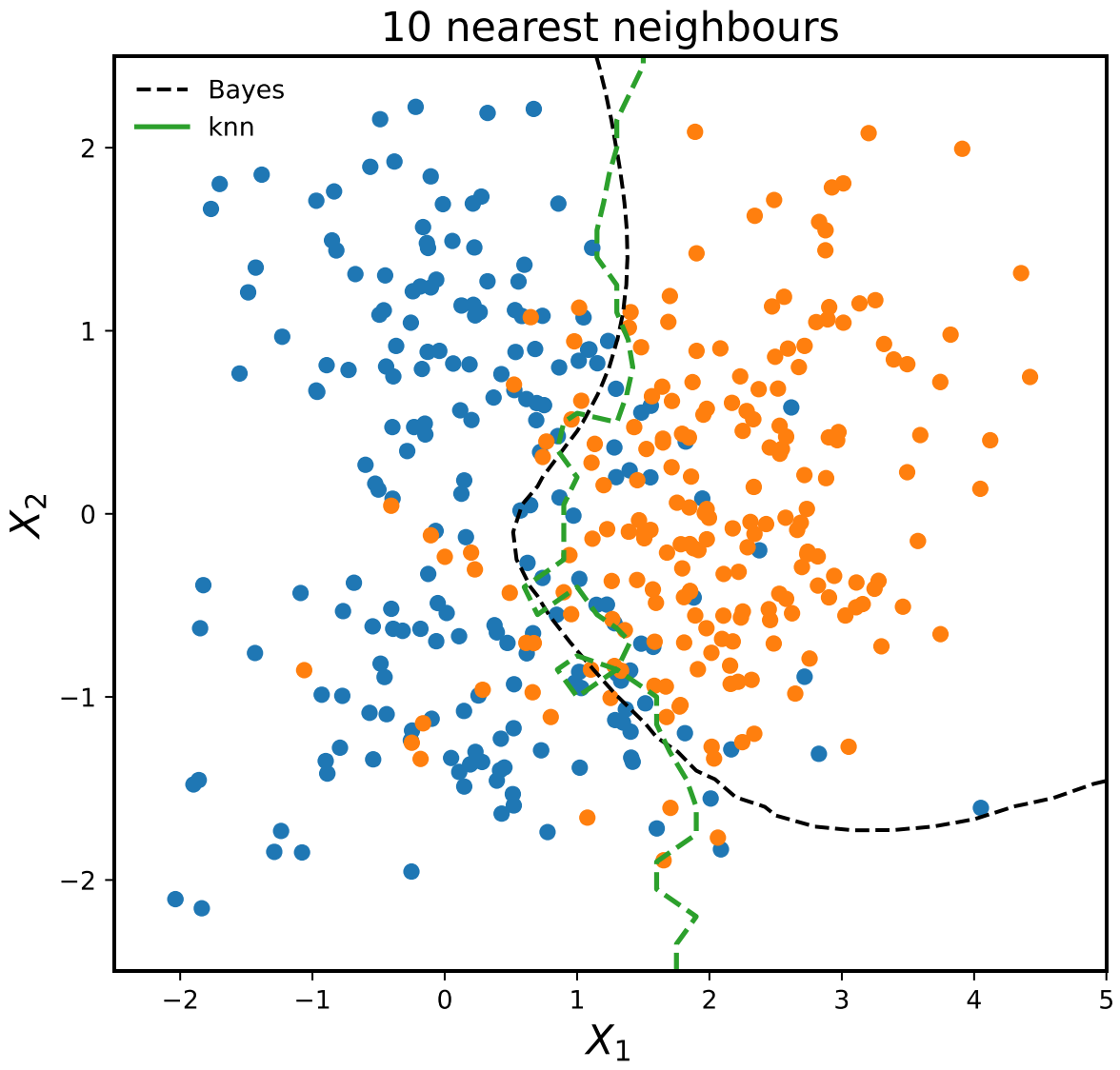
- **Logistic regression**

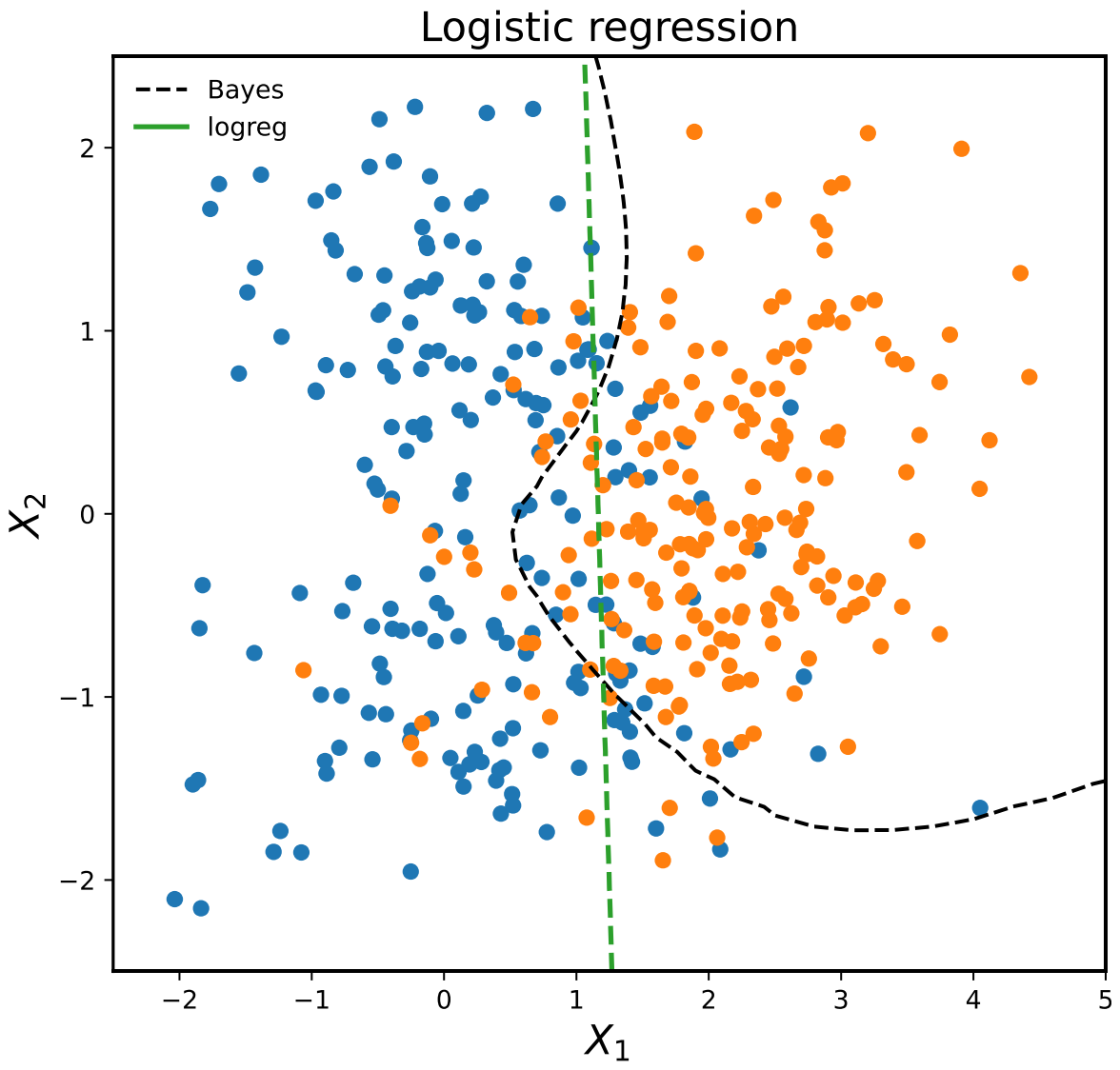
$$\mathrm{Prob}\Big(Y = 1 \;\Big|\; \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) \approx \sigma\big(\underbrace{\mathbf{w}^\top}_{\text{weight}}\mathbf{x}_0 + \underbrace{b}_{\text{bias}}\big)$$

$\sigma(\cdot)$ is a nonlinear activation function



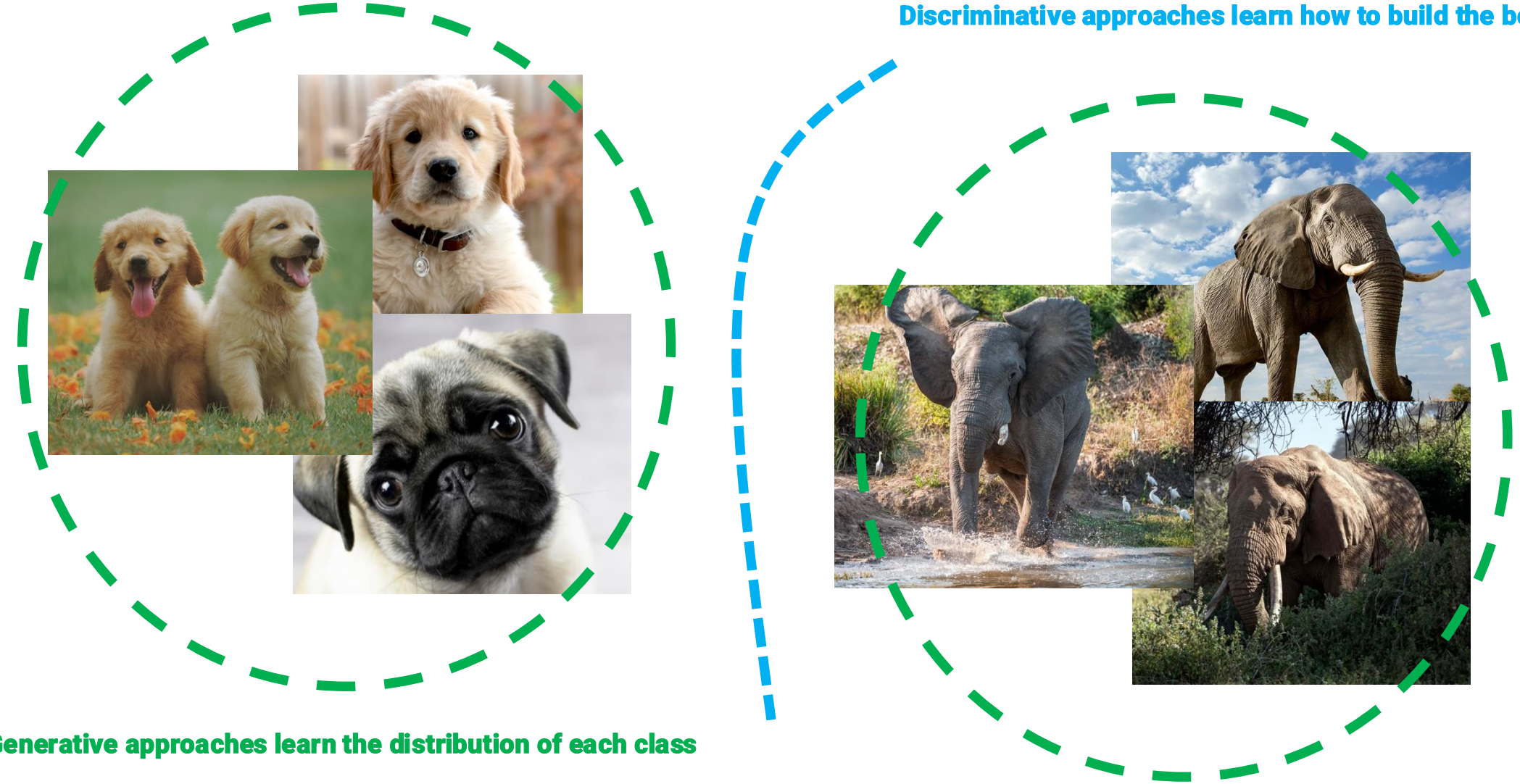
$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$


10 nearest neighbours
Bayes
knn
X_1
X_2


Logistic regression
Bayes
logreg
X_1
X_2

Generative approaches

Suppose we want to learn how to classify photos of dogs versus elephants



- Review from last lecture
- Generative approach to classification
- Comparisons between methods
- Final remarks on classification

Generative approaches

Instead of modeling the posterior distribution of the classes, we could model both the

- The prior distribution for each class $\text{Prob}(Y = k)$
- The conditional pdf of the predictors in each class $p(\mathbf{x} \mid Y = k)$

We can then use these quantities to define a classifier as follows

$$\boxed{f(\mathbf{x}_0) = \underset{k}{\text{argmax}}\ \text{Prob}\big(Y = k\big) \times p(\mathbf{x}_0 \mid Y = k)}$$

(note that it's the same thing as the Bayes classifier)

Different generative methods will model the **conditional pdf in a distinct way**

We will consider Gaussian discriminant analysis and the naïve Bayes classifier today

Generative approaches

- **Gaussian discriminant analysis**

In GDA, we suppose that the data from each class follows a Gaussian distribution

$$p(\mathbf{x} \mid Y = k) = \frac{1}{(2\pi)^{p/2}} \frac{1}{\det(\Sigma_k)^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\top \Sigma_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)\right)$$

Focusing on the two-class case and assuming that $\Sigma_0 = \Sigma_1 = \Sigma$ (shared covariance)

Linear discriminant analysis (LDA)

Two classes

$$\mathrm{Prob}\Big(Y = 1 \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) = \sigma(\mathbf{w}^\top \mathbf{x}_0 + b)$$

$$\mathbf{w} = \Sigma^{-1}(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)$$

$$b = -\frac{1}{2}\boldsymbol{\mu}_0^\top \Sigma^{-1} \boldsymbol{\mu}_0 + \frac{1}{2}\boldsymbol{\mu}_1^\top \Sigma^{-1} \boldsymbol{\mu}_1 + \log \frac{\mathrm{Prob}(Y = 0)}{\mathrm{Prob}(Y = 1)}$$

Generative approaches

- **Gaussian discriminant analysis**

In GDA, we suppose that the data from each class follows a Gaussian distribution

$$p(\mathbf{x} \mid Y = k) = \frac{1}{(2\pi)^{p/2}} \frac{1}{\det(\Sigma_k)^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\top \Sigma_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)\right)$$

Focusing on the two-class case and assuming that $\Sigma_0 = \Sigma_1 = \Sigma$ (shared covariance)

Linear discriminant analysis (LDA)

K classes

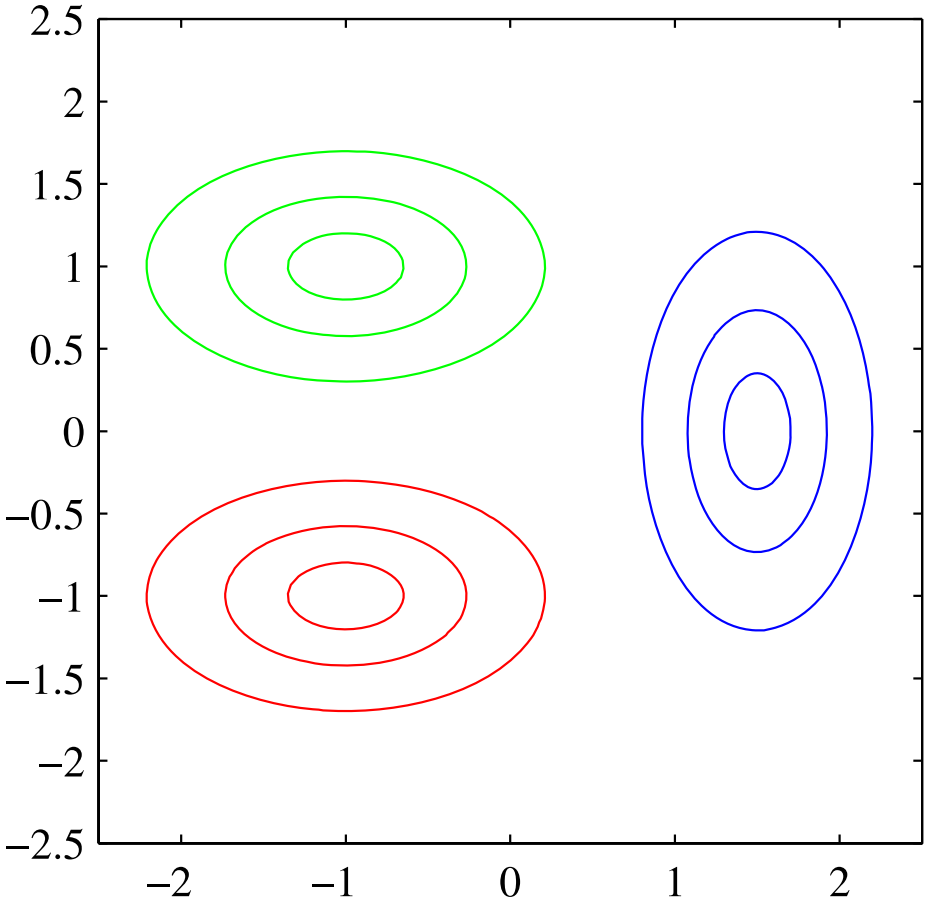
$$\mathrm{Prob}\Big(Y = k \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) = \frac{\exp(a_k(\mathbf{x}_0))}{\sum_j \exp(a_k(\mathbf{x}_0))}$$

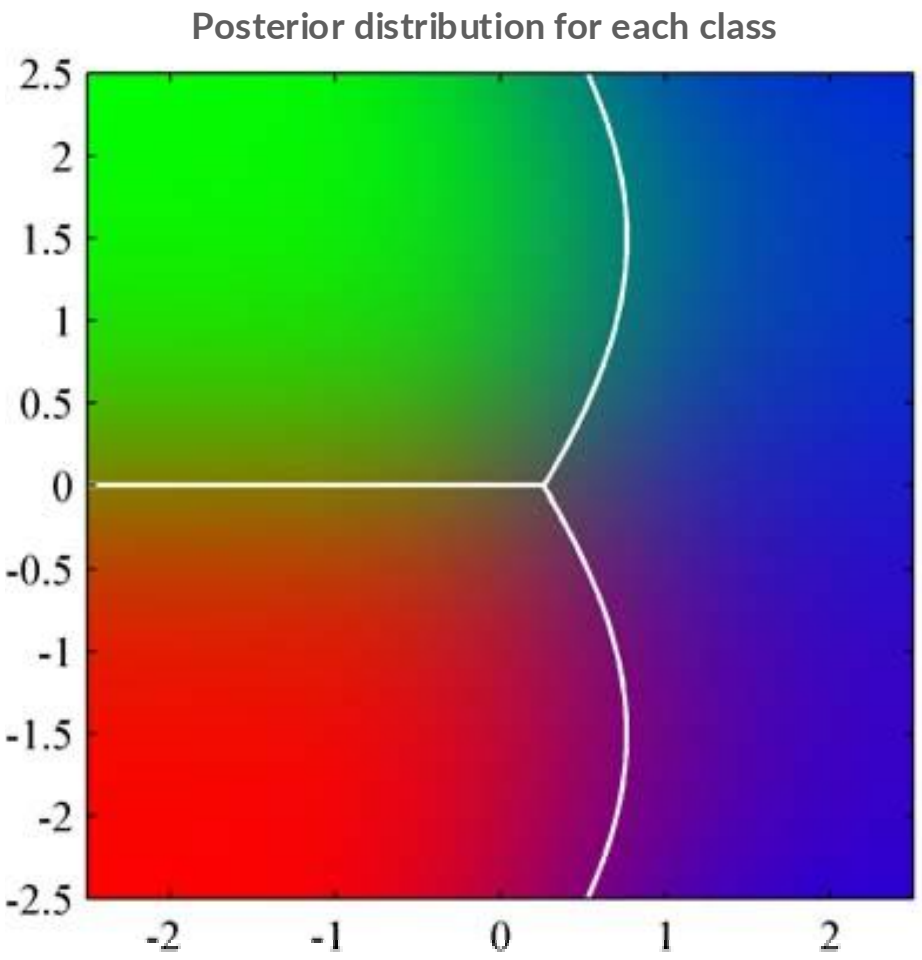
$$a_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x} + b_k$$

$$\mathbf{w}_k = \sigma^{-1} \boldsymbol{\mu}_k \quad \text{and} \quad b_k = -\frac{1}{2} \boldsymbol{\mu}_k^\top \Sigma^{-1} \boldsymbol{\mu}_k + \log \mathrm{Prob}(Y = k)$$

Generative approaches

- **Gaussian discriminant analysis**





Relaxing the shared covariance assumption gives quadratic discriminant analysis (QDA)

Generative approaches

- **Gaussian discriminant analysis**

  - LDA and logistic regression are similar but different: They both yield a linear boundary between classes, but LDA leverages information from the prior distribution, which is not available for the logistic regression
    - ↳ Note that when the data is truly Gaussian in each class, the classifiers are equivalent
  - Making assumptions over the structure of $p(\mathbf{x})$ can serve as a regularization in LDA
  - Gaussian discriminant analysis is, by definition, not adapted to qualitative predictors
    - ↳ We could, however, use Bernoulli or other discrete random variable to model the conditional pdfs as well
  - The parameters of logistic regression are estimated using an iterative optimization procedure based on MLE, whereas for LDA/QDA we can estimate directly from data

Generative approaches

- **Gaussian discriminant analysis**

When should we choose LDA or logistic regression?

If data is Gaussian, then the posterior is a logistic function. The converse is not true.

↳ Therefore, LDA makes stronger modeling assumptions about the data than logistic regression

When the assumptions are true, LDA will better fit the data than logistic regression

↳ LDA is asymptotically efficient: for large N, there's no better algorithm than LDA

By making weaker assumptions, logistic regression is more robust to deviations

Conclusion: it depends. But, in practice, logistic regression is used more often.

Generative approaches

- **Naïve Bayes classifier**

In GDA, we make strong assumptions about data and estimate $Kp^2 + Kp$ parameters

↳ For logistic regression we have $K(p+1)$ parameters to estimate

In Naïve Bayes, we assume that, for each class, the predictors are independent

$$p(\mathbf{x} \mid Y = k) = \prod_{i=1}^{p} p(x^i \mid Y = k) \quad \text{where} \quad \mathbf{x} = [x^1 \ \ldots \ x^p]^\top$$

We can, therefore, model just the marginal probabilities of the predictors

↳ If the marginals are Gaussian, then this means that we have $2Kp$ parameters to estimate

Do we actually expect the assumption of independence to be true? Not really...

↳ But when working with lots of predictors, this is the only possible approach!

Generative approaches

- **Naïve Bayes classifier**

The classifier can then be written as

$$f_{\mathrm{nB}}(\mathbf{x}_0) = \underset{k}{\operatorname{argmax}}\ \mathrm{Prob}\big(Y = k\big) \times \prod_{i=1}^{p} p(x_0^i \mid Y = k)$$

Since the marginals are independent, we can model each predictor differently

- Quantitative predictors can be modelled with Gaussian, Exponential, etc.
- Qualitative predictors can be modelled with Bernoulli, Dirichlet, etc.

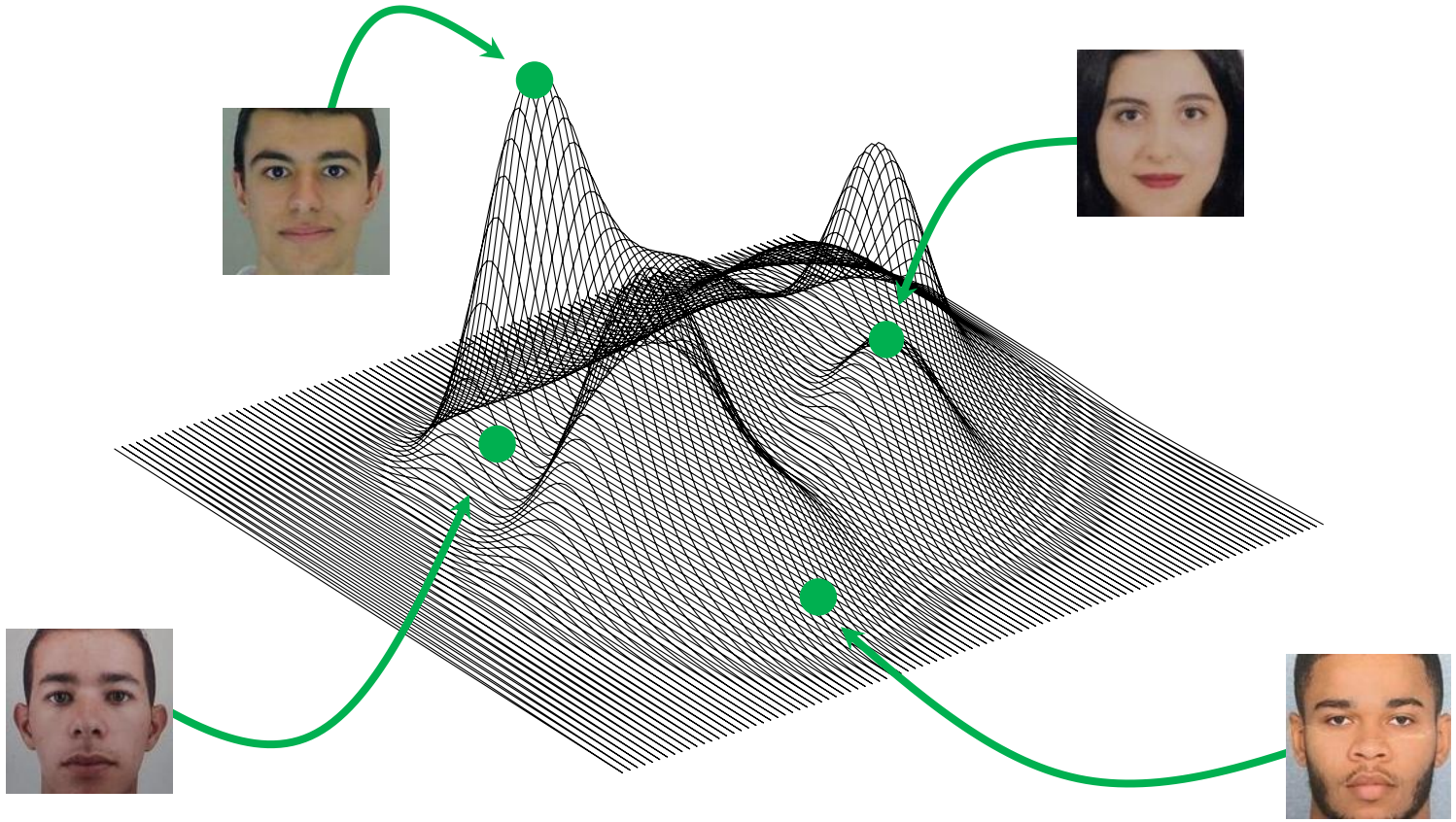
The Naïve Bayes classifier is often used in text classification, bioinformatics, etc.

Large dimensional predictors

Generative approaches

- **How do all this relate to "Generative AI" ?**

Current ML researchers try to model and/or learn the distribution $p(\mathbf{x})$ of a dataset

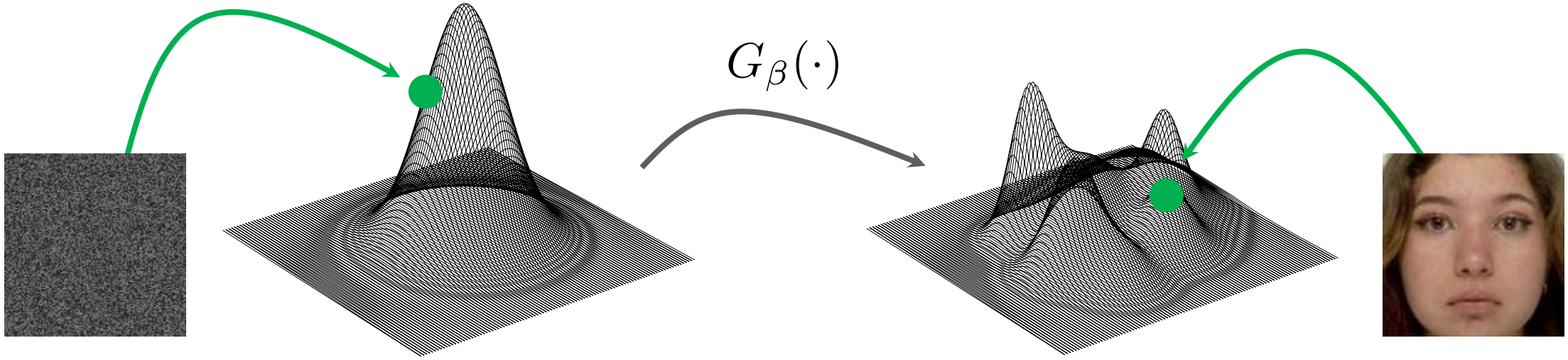


Generative approaches

- **How do all this relate to "Generative AI" ?**

Current ML researchers try to model and/or learn the distribution $p(\mathbf{x})$ of a dataset

Note, however, that it is often enough to **just know how to sample** from the distribution



$$\mathbf{z}_i \sim \mathcal{N}(0, \mathbf{I}) \qquad G_\beta(\mathbf{z}_i) = \mathbf{x}_i \sim p(\mathbf{x})$$

The challenge then is knowing how to build function $G_\beta(\cdot)$ -- deep neural networks!

- Review from last lecture
- Generative approach to classification
- Comparisons between methods
- Final remarks on classification

Comparisons between methods

We consider the simulated benchmark presented in James et al. (2022)

The benchmark compares the performance of six classifiers

KNN-1 : K-nearest neighbours (kNN) with K = 1

KNN-CV : kNN with K obtained with cross-validation

LDA : Linear discriminant analysis

Logistic : Logistic regression

NBayes : Gaussian naïve Bayes

QDA : Quadratic discriminant analysis

Under **six scenarios**: the first three have linear boundary, the other three are non-linear

Comparisons between methods

We consider the simulated benchmark presented in James et al. (2022)

Scenario 1

There were 20 training observations in each of two classes. The observations within each class were uncorrelated random normal variables with a different mean in each class.

$$\mathcal{D} = \{(\mathbf{x}_i, 0)\}_{i=1}^{N} \cup \{(\mathbf{x}_i, 1)\}_{i=1}^{N} \quad (N = 20)$$

$$\mathbf{x}_i \mid y = 0 \sim \mathcal{N}(\mu_0, \Sigma_0) \quad \text{and} \quad \mathbf{x}_i \mid y = 1 \sim \mathcal{N}(\mu_1, \Sigma_1)$$

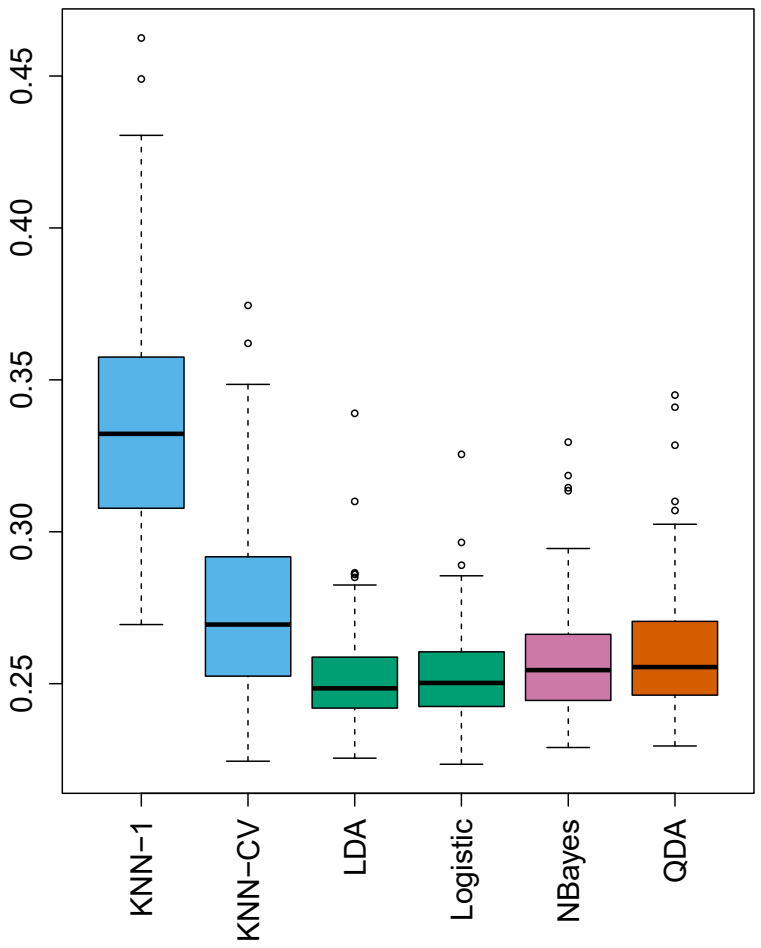
$$\mu_0, \mu_1 \in \mathbb{R}^2 \quad \text{with} \quad \mu_0 \neq \mu_1 \quad \text{and} \quad \underline{\Sigma_0 = \Sigma_1} = \begin{bmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{bmatrix}$$

This ensures linear boundary

(The generalization error of the classifiers were assessed on large test datasets with new simulated samples)

Comparisons between methods

Scenario 1



- LDA performed well in this setting, as one would expect since this is the model assumed by LDA.
- Logistic regression also performed quite well, since it assumes a linear decision boundary.
- KNN performed poorly because it paid a price in terms of variance that was not offset by a reduction in bias.
- QDA also performed worse than LDA, since it fit a more flexible classifier than necessary.
- The performance of naive Bayes was slightly better than QDA, because the naive Bayes assumption of independent predictors is correct.

Comparisons between methods

We consider the simulated benchmark presented in James et al. (2022)

Scenario 2 Same setup as Scenario 1, but predictors have correlation of -0.5

$$\mathcal{D} = \{(\mathbf{x}_i, 0)\}_{i=1}^{N} \cup \{(\mathbf{x}_i, 1)\}_{i=1}^{N} \quad (N = 20)$$

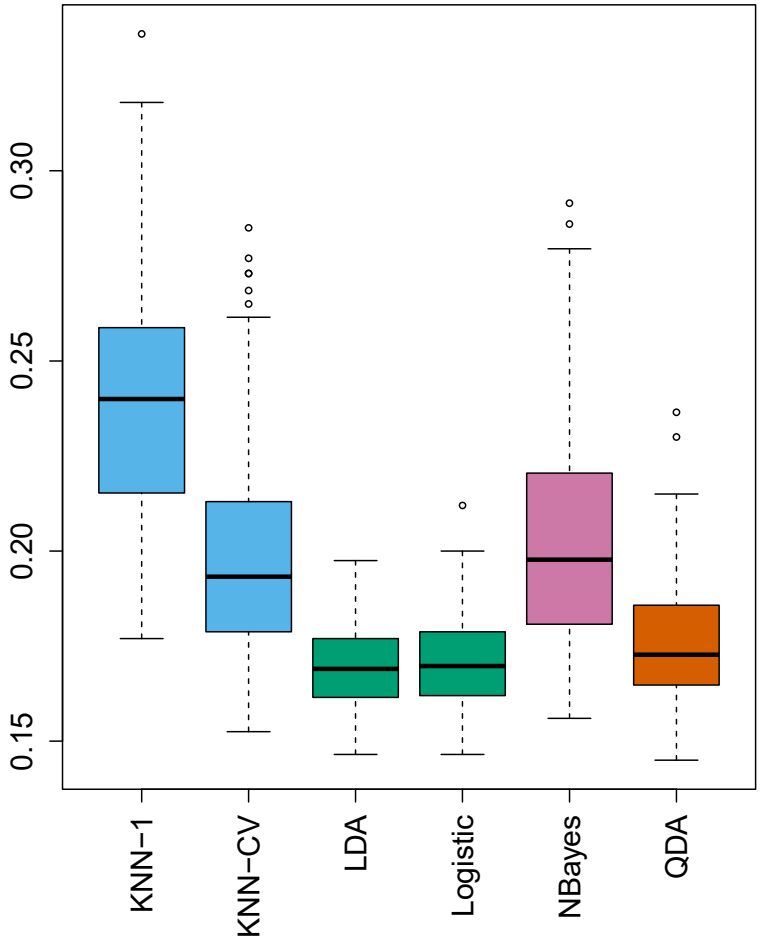
$$\mathbf{x}_i \mid y = 0 \sim \mathcal{N}(\mu_0, \Sigma_0) \quad \text{and} \quad \mathbf{x}_i \mid y = 1 \sim \mathcal{N}(\mu_1, \Sigma_1)$$

$$\mu_0, \mu_1 \in \mathbb{R}^2 \quad \text{with} \quad \mu_0 \neq \mu_1 \quad \text{and} \quad \underline{\Sigma_0 = \Sigma_1} = \begin{bmatrix} \sigma_1^2 & -\frac{1}{2}\sigma_1\sigma_2 \\ -\frac{1}{2}\sigma_1\sigma_2 & \sigma_2^2 \end{bmatrix}$$

This ensures linear boundary

Comparisons between methods

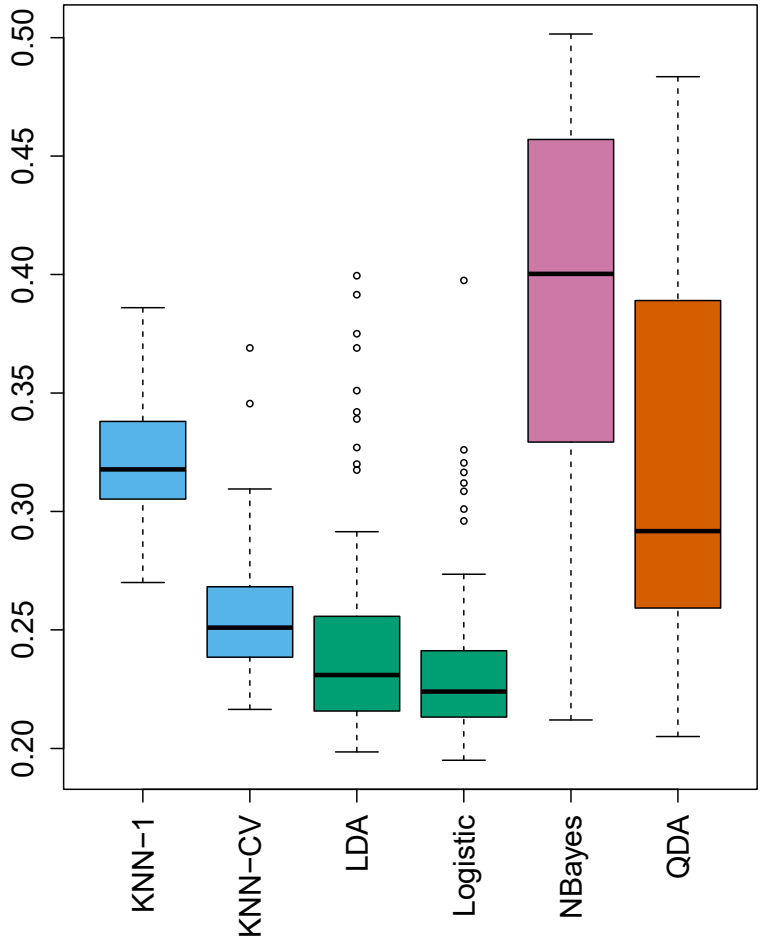
Scenario 2



- The performance of most methods is like the previous scenario. The notable exception is naive Bayes, which performs very poorly here, since the naive Bayes assumption of independent predictors is violated.

Comparisons between methods

Scenario 3



Same setup as in Scenario 2, but the predictors are generated from a t-distribution and N = 50

- The decision boundary is still linear, and so fit into the logistic regression framework. The set-up violated the assumptions of LDA, since the observations were not drawn from a normal distribution.
- Logistic regression outperformed LDA, and both methods were superior to the other approaches.
- In particular, the QDA results deteriorated considerably because of non-normality.
- Naive Bayes performed very poorly because the independence assumption is violated.

Comparisons between methods

We consider the simulated benchmark presented in James et al. (2022)

Scenario 4

Data is generated from a normal distribution with correlation 0.5 between the predictors in the first class, and correlation −0.5 between the predictors in the second class

$$\mathcal{D} = \{(\mathbf{x}_i, 0)\}_{i=1}^{N} \cup \{(\mathbf{x}_i, 1)\}_{i=1}^{N} \quad (N = 20)$$

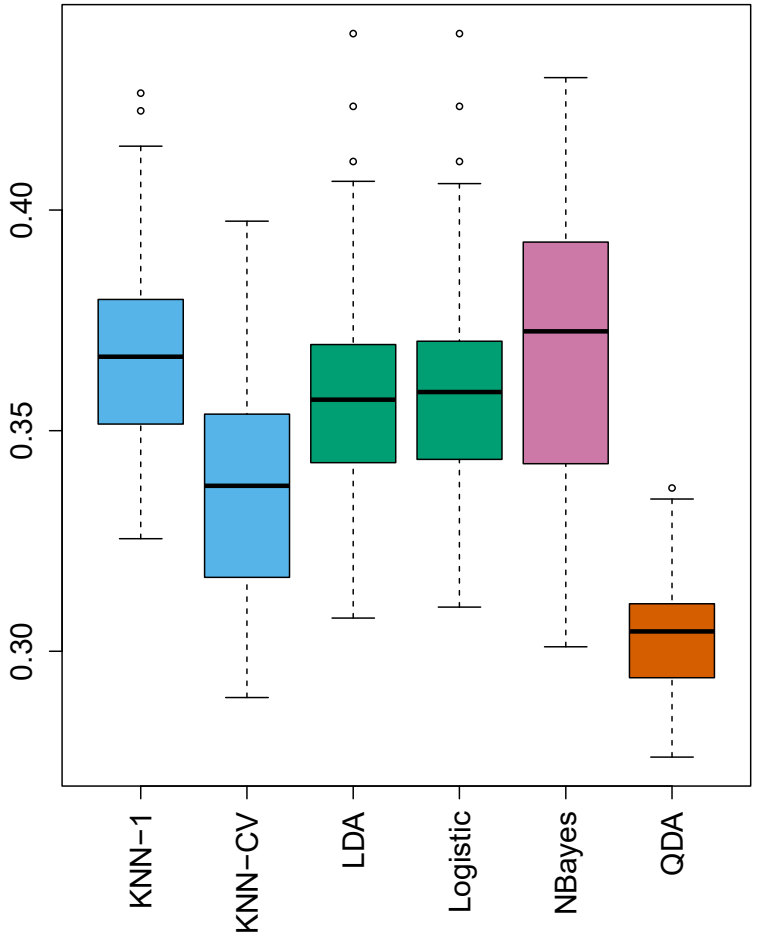
$$\mathbf{x}_i \mid y = 0 \sim \mathcal{N}(\mu_0, \Sigma_0) \quad \text{and} \quad \mathbf{x}_i \mid y = 1 \sim \mathcal{N}(\mu_1, \Sigma_1)$$

$$\mu_0, \mu_1 \in \mathbb{R}^2 \text{ with } \mu_0 \neq \mu_1 \text{ and } \Sigma_0 = \begin{bmatrix} \sigma_1^2 & \frac{1}{2}\sigma_1\sigma_2 \\ \frac{1}{2}\sigma_1\sigma_2 & \sigma_2^2 \end{bmatrix} \text{ and}$$

$$\Sigma_1 = \begin{bmatrix} \sigma_1^2 & -\frac{1}{2}\sigma_1\sigma_2 \\ -\frac{1}{2}\sigma_1\sigma_2 & \sigma_2^2 \end{bmatrix}$$

Comparisons between methods

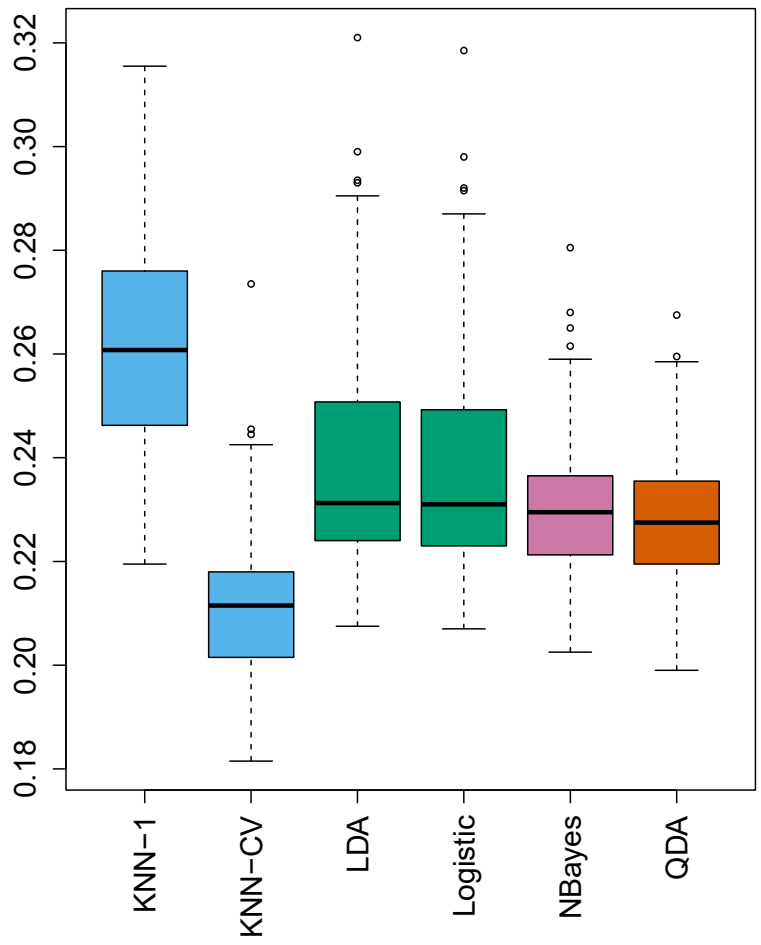
Scenario 4



- Setup fits QDA assumptions, which yields quadratic decision boundaries.
- We see that QDA outperformed all the other approaches.
- The naive Bayes assumption of independent predictors is violated, so naive Bayes performs poorly.

Comparisons between methods

Scenario 5

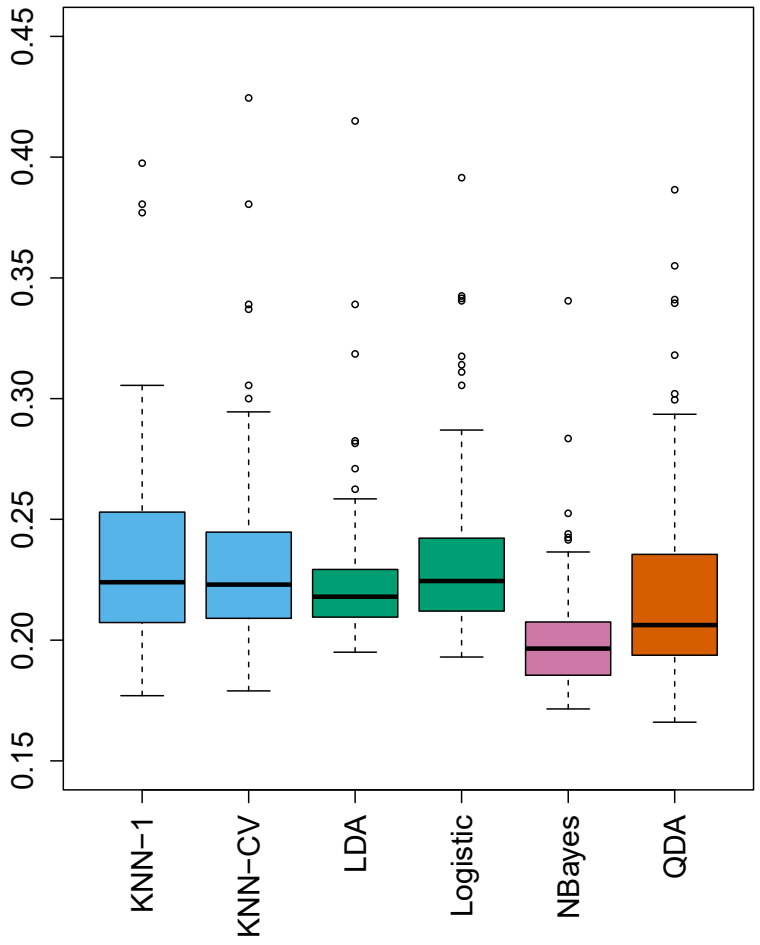


Data generated from Normal distribution with uncorrelated predictors. Then responses sampled from the logistic function applied to a non-linear function of the predictors.

- Both QDA and naive Bayes gave slightly better results than the linear methods,
- The much more flexible KNN-CV method gave the best results.
- But KNN with K = 1 gave the worst results out of all methods. This highlights the fact that even when the data exhibits a complex non-linear relationship, a non-parametric method such as KNN can still give poor results if the level of smoothness is not chosen correctly.

Comparisons between methods

Scenario 6



The observations were generated from a normal distribution with a different diagonal covariance matrix for each class. However, the sample size was very small: just N = 6 in each class.

- Naive Bayes performed very well, because its assumptions are met.
- LDA and logistic regression performed poorly because the true decision boundary is non-linear, due to the unequal covariance matrices.
- QDA performed a bit worse than naive Bayes, because given the very small sample size, the former incurred too much variance in estimating the correlation between the predictors within each class.
- KNN's performance also suffered due to the very small sample size.

Comparisons between methods

CONCLUSION

- We see that **no method dominates all** others in every situation.
- When the true decision boundaries are linear, the LDA and logistic regression approaches tend to perform well.
- When the boundaries are moderately non-linear, QDA or naive Bayes may give better results.
- For much more complicated decision boundaries, a non-parametric approach such as KNN can be superior. But the level of smoothness for a non-parametric approach must be chosen carefully.

- Review from last lecture
- Generative approach to classification
- Comparisons between methods
- Final remarks on classification

Final remarks

What if the data is not balanced between each class?

- Suppose you're doing your internship in a bank
- You train a classifier on data from customers so to detect when they will default

  i.e. not pay their debt
- You get a 2.75% error rate
- Should they hire you straightaway?

		True default status		
		No	Yes	Total
Predicted default status	No	9644	252	9896
	Yes	23	81	104
	Total	9667	333	10000

CONFUSION MATRIX

Final remarks

What are other approaches for classifications that we have not discussed?

Remember that the problem of classification can be split into two separate stages

Using discriminant functions bypasses the above procedure and ignores probabilities

↳ Support vector machines (SVM), perceptrons, and neural networks are examples

It is also very common to combine classifiers so to leverage their diversity

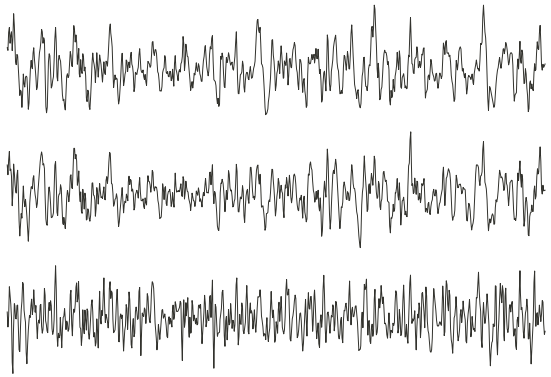
↳ Some strategies are random forests, bagging, and boosting

Final remarks

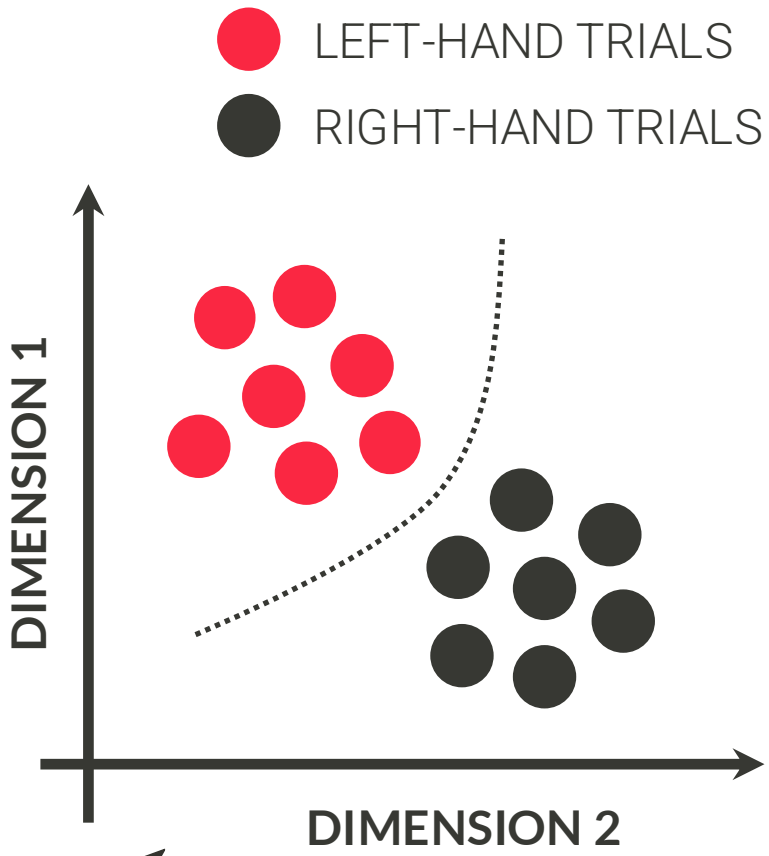
Feature engineering

Consider a brain-computer interface (BCI)

RECORDING



ENCODING

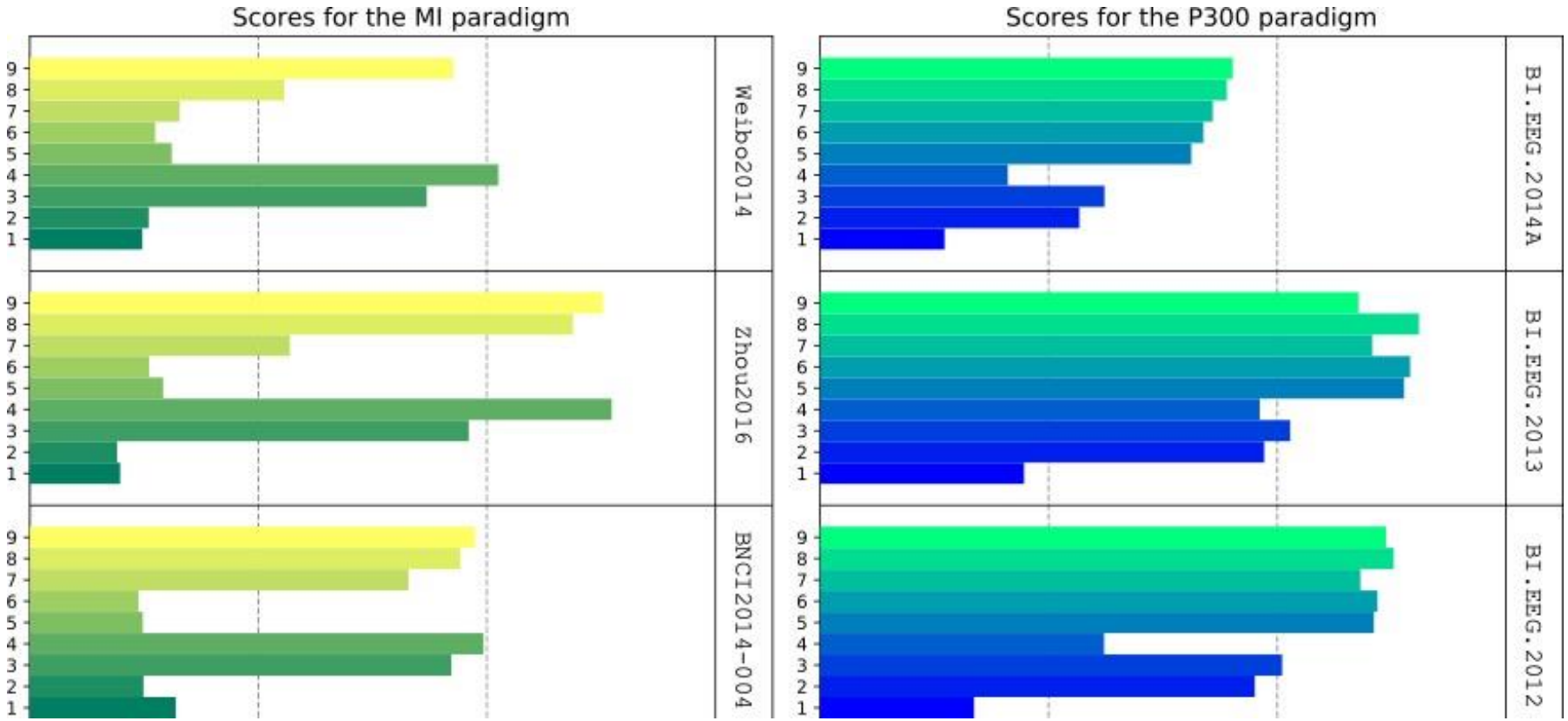


Final remarks

Feature engineering

Consider a brain-computer interface (BCI)

Different encoding strategies yield different performances



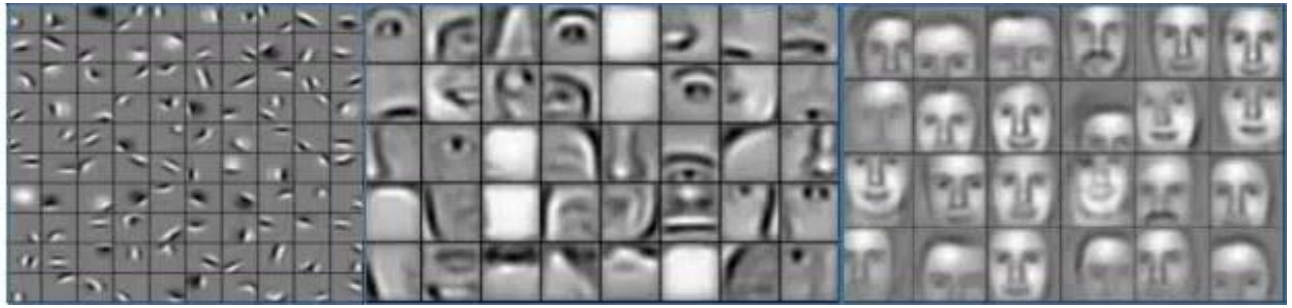
Final remarks

Feature engineering

The traditional workflow in machine learning is to have

but deep neural networks changed everything by learning features from data

Example of a convolutional neural networks learning features to represent images from a dataset



Layer 1 Layer 2 Layer 3

Final remarks

Differential programming

Recall the objective function for logistic regression

$$\mathcal{L}(\beta) = -\sum_i \Big(y_i \log(\pi_\beta(\mathbf{x}_i)) + (1 - y_i) \log\big(1 - \pi_\beta(\mathbf{x}_i)\big) \Big)$$

The parameters are obtained via a gradient descent : $\beta^{\text{new}} = \beta^{\text{old}} - \eta \nabla \mathcal{L}(\beta)$

This can be rather easily calculated by hand $\longrightarrow$ $\nabla \mathcal{L}(\beta) = \sum_i \big(y_i - \pi_\beta(\mathbf{x}_i)\big) \begin{bmatrix} 1 \\ \mathbf{x}_i \end{bmatrix}$

But what if we want to use a data-driven transformation on the data points?

$$\mathbf{x}_i \to g_\gamma(\mathbf{x}_i)$$

The gradients can start to become quite cumbersome to calculate…

Final remarks

Differential programming

A deep neural network is nothing more than this…

$$\mathcal{L}(\beta, \gamma_1, \ldots, \gamma_L) = -\sum_i \left(y_i \log\left(\pi_\beta\left(\mathcal{S}(\mathbf{x}_i)\right)\right) + (1 - y_i) \log\left(1 - \pi_\beta\left(\mathcal{S}(\mathbf{x}_i)\right)\right) \right)$$

Where

$$\mathcal{S}(\mathbf{x}_i) = g_{\gamma_L}\left(g_{\gamma_{L-1}}\left(\ldots\left(g_{\gamma_1}(\mathbf{x}_i)\right)\right)\right)$$

And the training follows something similar (not quite exactly) gradient descent…

Lucky for us, there has been a lot of development in **automatic differentiation**

PyTorch

TensorFlow

Final remarks

Transfer Learning

One of the main hypothesis of learning classifiers and regressors is that

The statistical distribution of data in training set	=	The statistical distribution of data in test set

In other words: “we train our tools on the same type of data on which we apply them”

But, what if this assumption is violated? Lots of research on the topic!

- Coping with statistical drifts in real data (e.g. subject fatigue)
- Green machine learning
- Explore invariant aspects of datasets